

CS 219 Operating Systems

Spring 2025-26

Mid-semester Exam (35 marks)

A

Name: _____ Roll No.: _____

Q1. State TRUE or FALSE and justify. [9 x 1 = 9 marks] **No correct justification, no marks.**

a.

*p = 23;

*p = 25;

Assume pointer p is defined.

The second line of instruction will **always** result in a TLB hit on access to address associated with pointer p.

FALSE.

An interrupt after the first line can cause the process to be switched out and another process or the kernel to be scheduled. In either case, if the virtual address ranges are overlapping the TLB will be flushed and on a reschedule of the first process the immediate access will be a TLB miss.

b. With the CoW (copy-on-write) optimization, the child process **always** shares the same page directory address with the parent process.

FALSE.

The page directory is the start (root page) of the page table, any write by the child or write process will have to propagate to change in the page directory and a separate page directory will be required for the child process.

c. After a call to a CoW-based **fork**, it is a good idea to schedule the child process **before** the parent process.

TRUE.

If a child process calls exec immediately after fork, then scheduling the parent before child process, can lead to unnecessary CoW handling page copies. All the pages will be copied to be discarded and address space overwritten by call to exec in the child process.

d. The **exec** system call does not modify a process's page table; it simply overwrites the contents of existing mapped pages.

FALSE.

The exec system call resets the current address space by copying/over writing contents of the new program to be loaded for execution. The size of the new address space depends on the size of the new program and hence may need change to the processes page table to reset unused page table entries.

- e. The PTE flags are only needed to identify swapped-out pages.

FALSE.

The MMU uses (bits of) the pte flags for several purposes — to determine validity of a pte (the valid bit), permission and access bits (R-read, W-write, X-execute, U-user, S-supervisor) for operations on the memory and mode of CPU execution for the same, updates the access and dirty bits on pte usage for read/write operations on the pointed to page.

- f. The page table of a process resides in its heap section.

FALSE.

A page table per-process metadata, used to store per-process virtual address to physical address mapping, and is a kernel object/variable and is stored as part of the kernel's memory region.

- g. **malloc'ed** memory that is not **free'd** will lead to a memory leak, even after the process exits.

FALSE.

On a process exit, the kernel can walk to the page table and clean up/free all mapped physical pages and the page table pages.

- h. The TLB entries of the *init* process are always flushed when init is descheduled.

TRUE.

The *init* process is yet another user space process, has its own address space and virtual to physical mappings, in its own page table. It is only special that it is the first user process (hand crafted by the OS). The init processes' virtual address mapping cannot be reused by other scheduled processes and hence TLB needs to be flushed.

- i. The IDTR (interrupt descriptor table register) is **not** updated on a process context switch.

TRUE.

The location of the interrupt handlers is OS-state/OS-meta data and the address of these functions does not depend on per-process execution/meta-data/run state etc. Moving around the interrupt handlers on a process switch or maintaining multiple copies (per-process) is unnecessary overhead.

Q2.

With 39-bit addressing (virtual addresses of size 39 bits), **2MB** physical pages are used for mapping virtual addresses. **Page table pages** are of size **4KB**, and the size of each PTE is **8 bytes**. —

(a) Draw a diagram to show the layout of the most compact page table and its usage.

(b) How to differentiate between **4KB** and **2MB** pages? **[3+1 = 4 marks]**

(a) Two level page table with offsets from the virtual address used in the following break up
9 : 9 : 21

$$\text{\#ptes/page} = 4096/8 = 512 \quad (\text{note that the page table pages are 4 KB})$$

(b) A bit in the pte-flag can be used to indicate whether it is pointing to a 4KB page or a 2MB page.

ptes in the first level page will have this bit set (encoded) to indicate pointing to a 4KB page

ptes in the second level page will have this bit reset to indicate pointing to a 2MB page

Note that ptes in the second level pages of the page table mapped to the mapped pages.

Q3. Consider the following program snippet (with line numbers added for clarity) —

[2 + 2 = 4 marks]

```
23.  // Some code is present above
24.  char *p = malloc (100 * sizeof (char));
25.  *p = 'O';           // O for Operating
26.  *(p + 10) = 'S';    // S for Systems
27.  free(p);
28.  // Some code is present below
```

a. Assume that **malloc** employs lazy page allocation (via **sbrklazy**), and Lines #25 and #26 above may not cause a page fault. Explain why.

- The lines before Line #25 could include a previous allocation of heap memory, and
- If this memory was subsequently accessed, and
- If the virtual addresses of that allocation and that used in Lines 25 and 26 are on the same page, the physical page is already mapped and hence will not result in page faults.

b. Explain the operation of the **free** call in Line #27.

Which actions are done in the user mode and which of them are done in the kernel mode?

In user space

- Verify if the pointer/address passed to free is valid start of malloc region
At an offset from the pointer (address) passed to free is the magic number indicating start of malloc'ed region
E.g., if ***(p-4) == MAGICNUM**
- If yes, use another offset on the passed address to determine size of the malloc'ed region
E.g., size = *(p-2);
- Invoke the system call to unmap the virtual addresses in the heap region
E.g., free(p-4, size+4) of sbrk (p-4, size+4, 0) // 0 indicates free
The adjustment of 4 bytes is to accommodate the malloc header.

In kernel mode

- Update process size and heap allocation meta-data to mark the corresponding virtual address space/region as free.
- Walk to the page table on a per-page basis to cover the VA space region and remove mappings in the page table and the last-level mapped page.
This has to be done a page-granularity level and if the va space occupies a partial

mapped page and other VA addresses are mapped in the same page, the ptes cannot be removed.

- Add free'd physical pages to the free pages list.

Q3. Consider the code given below. Assume that the code compiles with the appropriate header files. **[3 marks]**

<pre>int main() { pid_t pid = fork(); pid = waitpid(pid, &status, WNOHANG); while(pid == 0) { sleep(1); write(1, "A", 1); pid = waitpid(pid, &status, WNOHANG); } sleep(100); printf("%d\n", WEXITSTATUS(status)); exit(23); }</pre>	What is the output of this program? Explain. A is printed ~100 times Exit Status via child process is error/undefined 23 waitpid returns -1 (error) for child process and child process sleeps for 100s and then exits with value 23. waitpid returns 0 in parent process till child process terminates, after which child process is reaped and status value 23 is delivered to parent process.
---	--

Q4. Explain two differences with respect to segmentation and multi-level paging from the perspective of performance and overheads for a 32-bit address width. **[3 marks]**

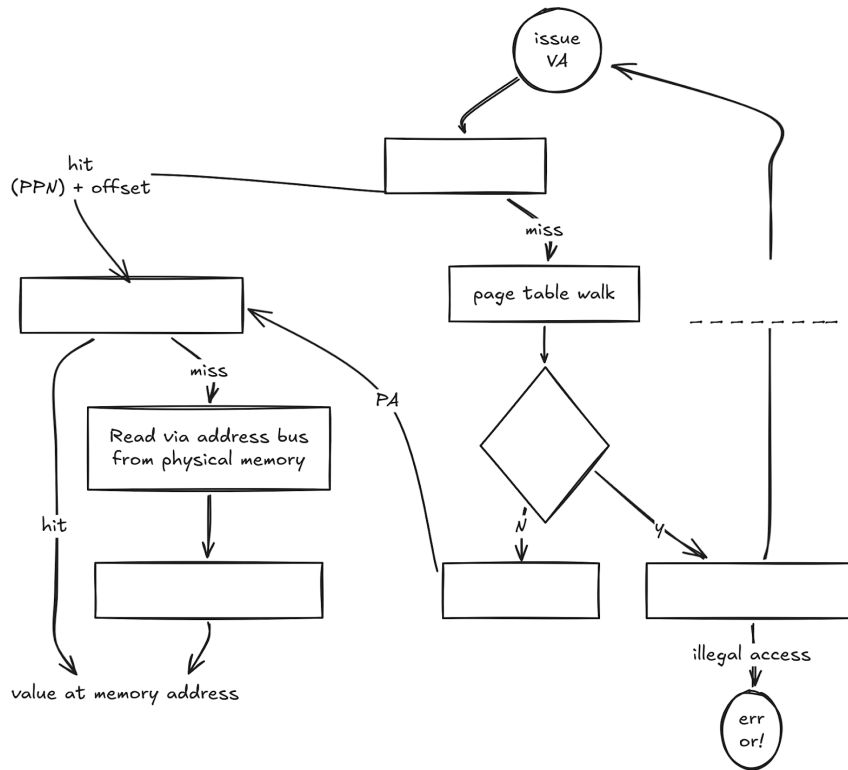
- (i) Segmentation has a single-level translation (for v2p) and with larger segments, takes fewer TLB entries for larger numbers of TLB hits and lower memory access latencies comparing to paging.

Virtual Address => Segment index => Segment base address + offset => Physical address
vs.

Virtual address => offset1 => pte => offset 2 => pte => => Physical address

- (ii) Fewer segments implies potentially larger segment sizes, which can lead to a large probability of internal fragmentation with segmentation. With paging, internal fragmentation can be restricted to sub-page level and maybe a few such pages.

Q5. Complete the following diagram representing the virtual-to-physical address translation based on the access path. Fill in all seven (empty) boxes and blanks. **[3 marks]**



- (i) Lookup TLB
- (ii) Lookup CPU cache
- (iii) Update CPU cache
- (iv) Page fault ?
- (v) Update TLB
- (vi) Page fault handler
- (vii) Re-issue instruction

Q6. The **fork** system call returns in two different processes.

Explain this statement with a diagram and the kernel objects/state that are used to make this happen. Start the discussion with the call in the user space and two returns in the user space.

[4 marks]

User Space

1. When a user program calls `fork()`:
2. The system call number corresponding to `sys_fork()` is placed in register `a7`.
3. The program executes an `ecall` (trap) instruction.
4. This causes a transition from user mode to kernel mode, transferring control to the kernel's trap handling mechanism.

Interrupt / Trap Handling

When the trap occurs:

1. the CPU switches to kernel mode.
2. Execution enters the trampoline code, which acts as a bridge between user space and kernel space.
3. In the trampoline: All user registers are saved into the process's trapframe and the page table is switched from the user page table to the kernel page table.
4. Control then jumps to `usertrap()`.

Inside `usertrap()`:

1. The kernel examines the cause of the trap.
2. Since the trap is due to a system call, it invokes the system call handler.
3. The system call handler reads register `a7` to determine which syscall to execute, and calls `sys_fork()`.

`sys_fork`

The `sys_fork()` function creates a new process by duplicating the parent.

1. Allocate a new PCB: The kernel finds a free Process Control Block (PCB) for the child process.
2. Set return address: The `ra` register in the child's context is set to the address of `forkret`.
3. Initialize PCB fields
 - a. Set PID, PPID, copy the address space size.
 - b. The page table of the parent is copied to the child.
 - c. The trapframe of the parent is copied to the child.
 - d. File descriptors of the parent are also copied.
4. Set return values
 - a. In the child's trapframe: `a0 = 0`
 - b. In the parent's trapframe: `a0 = child PID`
5. Make child runnable: The child's state is set to `RUNNABLE`, allowing the scheduler to schedule it.

After `sys_fork()` returns:

1. The parent process continues execution through the normal syscall return path.
2. The child process, when scheduled, begins execution at forkret because its ra register was set to that address.

Returning to user space

Parent Process:

1. After sys_fork, control returns to the syscall handler.
2. Execution continues in usertrap().
3. usertrap() prepares the trapframe for returning to user mode.
4. Control jumps back to the trampoline.
5. The trampoline:
 - a. Restores all registers from the trapframe
 - b. Switches to the user page table
 - c. Executes the return to user space
6. Thus, the parent process resumes execution after fork(), with return value child PID.

Child Process:

When the scheduler selects the child process, execution begins at forkret.

1. forkret performs steps similar to the return path of usertrap:
2. It prepares the kernel state in the trapframe.
3. It jumps to the trampoline code.
4. The trampoline then:
 - a. Restores all registers
 - b. Switches to the child's page table
 - c. Returns to user space
5. Thus, the child process also resumes execution after fork(), but with return value 0.

Q7. Design a simple and low-overhead approximate LRU (least recently used) policy for page evictions. The technique must not use per-page counters. Explain using a flowchart.
[3 marks]

Answer 1

- All physical pages that are not mapped to kernel virtual addresses used by kernel code and kernel data are considered for eviction.
- Assume each mapped physical page is arranged in some linear order/list.
- Pages are referred to by their PTEs.
- the PTE of a page consideration is checked,

if the ACCESSED bit is set, the bit is reset to and the next page is considered.
else if the ACCESSED bit is not set, the page is evicted

Answer 2

- All mapped physical pages are arranged in two lists — active and inactive lists.
- Newly mapped pages are added to the head of the active list.

- For eviction pages are removed from the tail of the inactive list.
- Pages are referred to by their PTEs.
- The active list is scanned from the tail of the list.
- If the ACCESSED bit of a page in the ACTIVE list is set, the bit is reset and next page is considered.
- Else if ACCESSED bit is reset, the page is moved to head of the INACTIVE list
- During eviction pages from the tail of the inactive list with ACCESSED bits set to zero are evicted.
- Pages in the inactive list with ACCESSED bits set are moved to head of the active list.

Q8. Explain **two** undesirable side effects of swapping pages mapped to the kernel virtual address space. **[2 marks]**

- (i) Assume the page mapped to the kernel virtual address range on which the interrupt handler table or the interrupt handler function is resident has been swapped out.
There is no way to run kernel logic to handle an interrupt, as the interrupt handler and logic to handle swapped pages is swapped out!
- (ii) Assume if the memory region where kernel objects are stored is swapped out. Since kernel objects are needed often, e.g., PCBs, page table pages etc., handling them via swapping will add to the indirection delay and kernel overhead to provide the abstraction/service.

Rough work